

CML 2 - Data Preparation for ML - Class notes

Offline 2h. Keep energy on **votes**, **pair talk**, and “**fit on train only**.”

0. Hook (5 min)

Show a tiny table with salary + city + “hired?” and a suspiciously perfect accuracy.

Ask the room: “Would you ship this model?” Hands up / down.

Line to land: “Prep mistakes don’t look like code bugs - they look like *fake* good scores.”

1. Why data prep sits before the model

Pipeline sketch on board:

raw table -> clean / impute -> encode -> scale -> **model** -> evaluate

Interactive: students shout which step they think is skipped most often (usually scale or split hygiene).

2. Train / test hygiene - split first

Visual: visuals/01_split_first.svg

Rule: any statistic learned from data (mean, std, category list, median fill) must be learned from **train only**, then applied to test.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify if class imbalance
)
```

Stuck point: “I scaled the whole CSV then split” feels efficient - it’s leakage.

3. Leakage patterns

Visual: visuals/02_leakage.svg

Activity: activities.md - Leakage detective (3 scenarios)

Pattern	Example	Fix
Fit-on-all	Scaler / imputer / encoder fit on full X	Fit on train; transform train + test
Target leakage	Feature that only exists after the label is known	Drop or redefine feature as-of prediction time
Temporal peek	Future rows in train when predicting past	Time-based split

Line to land: “Ask: *at prediction time, would I already have this number?*”

4. Encoding categoricals

Visual: visuals/03_encoding.svg

Kind	Prefer	Avoid
Nominal (city, color)	One-hot / dummy	Treating as integers with order
Ordinal (S/M/L, rating)	Ordered integer / ordinal encoder	One-hot if order matters for the model story

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
# Fit encoder on train categories only; handle_unknown='ignore' for new test levels
```

Interactive: hold up cards - “Red / Blue / Green” vs “Low / Med / High” - room picks encoding.

5. Scaling numerics

Visual: visuals/04_scaling.svg

Why: distance / gradient methods treat large-range features as more important.

Scaler	Idea	When
Standard (z-score)	mean 0, std 1	Most defaults; approx Gaussian
MinMax	squash to [0,1]	Bounded features; neural-ish ranges
Robust	median / IQR	Heavy outliers

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(X_train[num_cols])
X_train_s = scaler.transform(X_train[num_cols])
X_test_s = scaler.transform(X_test[num_cols]) # same mean/std - do NOT refit
```

Demo beat: two points with age vs income - distance before/after scale (board sketch).

Note for later CML: trees often need less scaling; still teach the habit now.

6. Missing values

Visual: visuals/05_missing.svg

Strategies (compare, don’t memorize one “best”):

1. Drop rows (ok if few missings, MCAR-ish)
2. Drop column (mostly empty / useless)
3. Impute mean / median / mode
4. Impute + **missing indicator** flag
5. Domain fill (0 for “no prior purchases”)

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median").fit(X_train[num_cols])
```

Interactive: 60-second pair debate - “median fill vs drop rows” for 5% missing vs 40% missing.

7. Glue it: preprocessing as a pipeline

Line to land: “Pipeline = the only way to not forget fit-on-train.”

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())]), num_cols),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("oh", OneHotEncoder(handle_unknown="ignore"))]), cat_cols),
])

clf = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=1000))])
clf.fit(X_train, y_train)
clf.score(X_test, y_test)
```

Ritual on board: draw a locked box “TRAIN STATS” - only train opens it; test only borrows.

8. Close

- Exit ticket: quizzes.md
- Preview next: Linear Regression deep dive (Session 3) - scaling will show up again in GD stability